Monte Carlo Tree Search - Final Report

MCTS-SINDy: Monte Carlo Tree Search for Constructing SINDy Libraries

Discovering Physical Laws from Data

[Link to GitHub Repository]

Gabriella FERNANDES MACEDO[1] and Ruben IFRAH[1]

[1]Master IASD, Dauphine-PSL

April 5, 2026

Abstract

Discovering the governing equations of nonlinear dynamical systems from data requires balancing mathematical expressiveness with optimization efficiency. Standard data-driven methods, such as the Sparse Identification of Nonlinear Dynamics (SINDy), rely on predefined function libraries that suffer from combinatorial explosion and fundamentally fail to capture unmapped, deeply nested physical topologies. Conversely, purely symbolic approaches like the Symbolic Physics Learner (SPL) offer potentially infinite structural flexibility but encounter severe computational bottlenecks when optimizing continuous parameters. In this report, we propose MCTS-SINDy, a hybrid architecture that leverages Monte Carlo Tree Search (MCTS) for unconstrained symbolic exploration while utilizing SINDy's sparse regression as a solver for instantaneous coefficient optimization. To overcome the reward entanglement problem inherent in full-library generation, our model maps the search to a greedy, iterative additive process, evaluating isolated mathematical features sequentially. Furthermore, we enforce strict structural parsimony using a Bayesian Information Criterion (BIC) reward threshold, and replace standard uniform random rollouts with Sequential Halving Using Scores for Search (SHUSS), yielding a 50x speedup in simulation efficiency. Empirical benchmarking across linear oscillators, the chaotic Lorenz attractor, and deeply nested systems demonstrates that our architecture respects Occam's Razor principle, successfully isolating exact governing dynamics in difficult cases.

Contents

1 Introduction **3**

2 Problem setup **3**

3 Relevant contributions leading to our work **4**
3.1 Sparse identification of nonlinear dynamical systems (SINDy) 4
3.2 Symbolic Physics Learner (SPL) 6

4 The main challenge: generating the library and feature entanglement **7**

5 Proposed architecture: iterative MCTS **7**
5.1 State representation and grammar 8
5.2 The greedy additive loop 9

6 Solving the MCTS bottlenecks **10**
6.1 Designing the reward 10
6.1.1 The baseline SPL formulation 11
6.1.2 Intermediate Additive Penalty (using SINDy L_0 Norm) 11
6.1.3 The Information Theory perspective (using BIC) 11
6.1.4 Temperature bound and nested feature regularization 11
6.2 Parsimony threshold and Occam's razor 12
6.3 Simulation efficiency: from uniform rollouts to SHUSS 12

7 Detailed Case Study: The Colpitts Chaos Oscillator **13**

8 Algorithm Benchmarking **14**
8.1 Experimental setup and metrics 14
8.2 Linear baseline: Damped Harmonic Oscillator 15
8.3 Chaotic systems: The Lorenz Attractor 16
8.4 Deep nesting equation 17

9 Conclusion and future Work **18**

1 Introduction

Discovering mathematical expressions that describe physical phenomena from data is a challenging problem at the intersection of machine learning and scientific modeling. In many situations, the underlying equations governing a system are unknown and must be inferred directly from observed data. This task becomes particularly difficult for nonlinear systems, where the space of possible expressions grows rapidly and exhibits a highly complex structure.

Several approaches have been proposed to address this problem. Methods based on sparse regression aim to identify simple models from a predefined set of candidate functions, while symbolic regression explores more flexible representations by combining mathematical operators into structured expressions. However, these approaches often rely on strong prior assumptions or suffer from scalability issues as the search space increases.

In this project, we investigate an approach based on Monte Carlo Tree Search (MCTS) to address the discovery of governing equations from data. Rather than relying solely on predefined function libraries or directly searching for complete equations, we explore how tree-based search methods can be used to navigate the space of candidate mathematical expressions in a structured manner.

The objective is to combine the flexibility of symbolic exploration with the robustness of data-driven identification methods in order to discover accurate and interpretable models of dynamical systems.

The remainder of this report is structured as follows. In Section 2, we formally define the problem of discovering governing equations from data. Section 3 reviews relevant contributions, including Sparse Identification of Nonlinear Dynamical Systems (SINDy) and the Symbolic Physics Learner (SPL). Section 4 discusses the main challenge of generating candidate features and handling feature entanglement. In Section 5, we present our proposed iterative MCTS-based architecture, including the greedy additive loop and the state representation with grammar. Section 6 addresses key bottlenecks in MCTS, such as reward design, parsimony thresholds, and simulation efficiency improvements using SHUSS. Finally, Section 7 presents a detailed case study that is completed with experimental results and benchmarking on a variety of dynamical systems in Section 8, and Section 9 concludes with a discussion of future work.

2 Problem setup

We consider the problem of discovering the governing equations of a dynamical system from observed data. We assume that the system can be described by an ordinary differential equation of the form:

$$\dot{x}(t) = f(x(t)), \tag{1}$$

where $x(t) \in \mathbb{R}^n$ denotes the state of the system at time t, and f is an unknown function governing the system dynamics.

In practice, the explicit form of f is not available. Instead, we observe sampled trajectories of the system:

$$\{x(t_1), x(t_2), \ldots, x(t_m)\}, \tag{2}$$

from which time derivatives $\dot{x}(t)$ can either be measured or estimated numerically.

The objective is to recover an explicit and interpretable expression for f from these observations. This problem can be viewed as a regression task over a large and structured space of candidate functions

A common approach consists in approximating f as a linear combination of candidate functions:

$$\dot{X} \approx \Theta(X)\,\Xi \tag{3}$$

where $\dot{X}$ is the time derivative data, $\Theta(X)$ is a library of candidate features (e.g., polynomials, trigonometric functions), and Ξ is a sparse vector of coefficients.

Two main challenges arise in this setting. First, the space of possible candidate functions is extremely large, especially when allowing nonlinear and compositional expressions. Second, the model must achieve a balance between accuracy and interpretability, avoiding overly complex expressions that do not generalize well.Addressing these challenges requires both an efficient exploration of the space of candidate expressions and a robust method for selecting a small number of relevant terms.

3 Relevant contributions leading to our work

Some methods trying to address the different challenges of this problem already exist. In the next section we will present the ones that inspired us in our project from different perspectives. Indeed, SINDy focuses on efficient sparse regression within a predefined function space, while SPL explores the space of expressions more freely using a tree-based search guided by MCTS.

3.1 Sparse identification of nonlinear dynamical systems (SINDy)

Sparse identification of nonlinear dynamical systems (SINDy) [2] is a method designed to automatically discover the governing equations of a dynamical system from observed data. The central idea is that, in many physical systems, only a few terms in the equations are truly significant, making the model sparse in the space of possible functions.

The approach begins by collecting time-series data of the system's state $x(t)$ and its time derivative $\dot{x}(t)$.

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^T(t_1) \\ \mathbf{x}^T(t_2) \\ \vdots \\ \mathbf{x}^T(t_m) \end{bmatrix} = \overbrace{\begin{bmatrix} x_1(t_1) & x_2(t_1) & \cdots & x_n(t_1) \\ x_1(t_2) & x_2(t_2) & \cdots & x_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(t_m) & x_2(t_m) & \cdots & x_n(t_m) \end{bmatrix}}^{\text{state}} \Bigg\downarrow \text{time} \tag{4a}$$

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{\mathbf{x}}^T(t_1) \\ \dot{\mathbf{x}}^T(t_2) \\ \vdots \\ \dot{\mathbf{x}}^T(t_m) \end{bmatrix} = \begin{bmatrix} \dot{x}_1(t_1) & \dot{x}_2(t_1) & \cdots & \dot{x}_n(t_1) \\ \dot{x}_1(t_2) & \dot{x}_2(t_2) & \cdots & \dot{x}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{x}_1(t_m) & \dot{x}_2(t_m) & \cdots & \dot{x}_n(t_m) \end{bmatrix} \tag{4b}$$

A library of candidate functions $\mathbf{\Theta}(\mathbf{X})$ is then constructed to represent potential terms in the equations. For example, $\mathbf{\Theta}(\mathbf{X})$ may consist of constant, polynomial and trigonometric terms:

$$\mathbf{\Theta}(\mathbf{X}) = \begin{bmatrix} | & | & | & | & | & | & | & | & | \\ \mathbf{1} & \mathbf{X} & \mathbf{X}^{P_2} & \mathbf{X}^{P_3} & \cdots & \sin(\mathbf{X}) & \cos(\mathbf{X}) & \sin(2\mathbf{X}) & \cdots \\ | & | & | & | & | & | & | & | & | \end{bmatrix} \tag{5}$$

where $\mathbf{X}^{P_2}$ denotes the nonlinearities of degree 2 in the state variable $\mathbf{X}$, given by:

$$\mathbf{X}^{P_2} = \begin{bmatrix} x_1^2(t_1) & x_1(t_1)x_2(t_1) & \cdots & x_2^2(t_1) & x_2(t_1)x_3(t_1) & \cdots & x_n^2(t_1) \\ x_1^2(t_2) & x_1(t_2)x_2(t_2) & \cdots & x_2^2(t_2) & x_2(t_2)x_3(t_2) & \cdots & x_n^2(t_2) \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ x_1^2(t_m) & x_1(t_m)x_2(t_m) & \cdots & x_2^2(t_m) & x_2(t_m)x_3(t_m) & \cdots & x_n^2(t_m) \end{bmatrix} \tag{6}$$

Sparse regression techniques are applied to select only the most relevant terms from this library. The result is a model that achieves a trade-off between accuracy and interpretability while avoiding overfitting:

$$\dot{\mathbf{X}} \approx \mathbf{\Theta}(\mathbf{X})\,\mathbf{\Xi} \tag{7}$$

where $\dot{\mathbf{X}}$ is the time derivative data, $\mathbf{\Theta}(\mathbf{X})$ is a library of candidate features and $\mathbf{\Xi}$ is a sparse vector of coefficients which is obtained by regression techniques such as Sequentially Thresholded Least Squares to select only the most relevant terms.

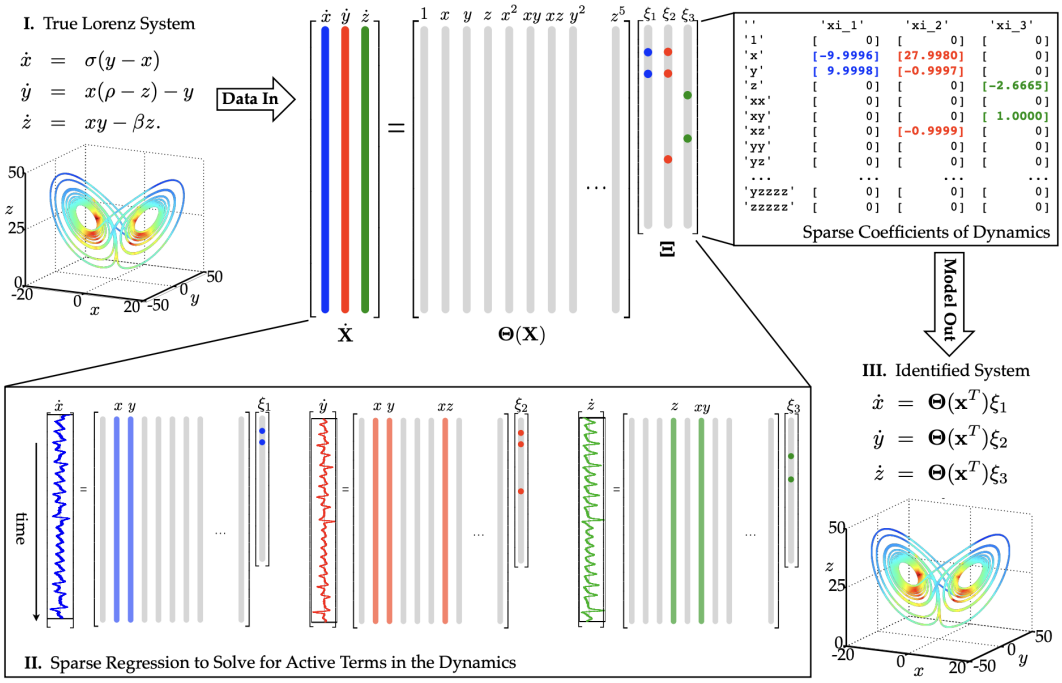

Figure 1: Schematic of the SINDy algorithm, demonstrated on the Lorenz equations. Taken from [2].

SINDy has been successfully applied to a wide range of systems, from simple linear and nonlinear oscillators to the chaotic Lorenz system and fluid vortex dynamics, demonstrating its ability to efficiently uncover underlying physical laws.

While SINDy provides an efficient framework for discovering governing equations from data, its performance strongly depends on the choice of a predefined function library. Assumptions must be made in advance, and any term not included in this library cannot be discovered. This limits the flexibility of the method, especially for complex or unknown systems.

More precisely, the complexity of constructing a library grows exponentially with the number of basis function. For instance, for a polynomial basis with n variables and degree d, the number of candidate terms is $\binom{n+d}{d}$. If we compute this with $n = 3$, it already increases from 10 terms at degree 2 to 56 terms at degree 5. This rapid growth becomes even more pronounced when additional nonlinear functions are included. Thus we cannot simply include an exhaustive list of relevant basis functions and expect the model to perform well.

Another unresolved problem of SINDy is its linear formulation. This makes it such that, to recover nonlinear structures such as $sin(xy)$, one has to include it in the library. The space of possible candidate functions being extremely large when allowing nonlinear and compositional expressions, we face the challenge of constructing nonlinear features for the model.

3.2 Symbolic Physics Learner (SPL)

In contrast to SINDy, which relies on a predefined library of candidate functions, the Symbolic Physics Learner (SPL) [5] adopts a more flexible approach to discovering governing equations from data. SPL represents mathematical expressions as expression trees, where each node corresponds to either a variable, a constant, or a mathematical operator. This tree-based representation allows for constructing a wide variety of candidate equations without being restricted to a fixed function library.

To explore the space of possible expressions efficiently, SPL uses a UCT-based Monte Carlo Tree Search (MCTS) algorithm. It iteratively expands the search tree, simulates candidate equations, and evaluates them based on a reward function that balances accuracy and sparsity. In other words, SPL not only seeks equations that fit the data well but also favors shorter formulas, avoiding unnecessary complexity. Specifically, the reward r for a candidate expression $\hat{f}$ is defined as:

$$r = \frac{\eta^n}{1 + \mathrm{RMSE}} \tag{8}$$

where RMSE is the root-mean-square error between the predicted and observed state derivatives, $\eta \in (0, 1)$ is a discount factor, and n represents the total number of production rules utilized to construct the parse tree. In other words, SPL not only seeks equations that fit the data well (by minimizing the denominator) but also favors shorter formulas by exponentially decaying the reward as the grammatical complexity n increases, hence avoiding unnecessary complexity.

Several modifications improve the classical MCTS for equation discovery. First, SPL uses the maximum reward rather than the expected reward, which better aligns with the objective of identifying the best expression. Second, it employs adaptive scaling to reduce uncertainty in the reward caused by errors in estimating system derivatives. Third, highly promising subtrees are reused in future searches, accelerating convergence toward high-quality expressions.

Despite these strengths, SPL suffers from a critical computational bottleneck: it relies on continuous parameter optimization to determine the numerical constants embedded within candidate expressions. Whenever the MCTS agent generates a structural equation skeleton containing constant placeholders (e.g., $c_1 \sin(c_2 x) + c_3$), an iterative numerical solver (such as BFGS or the Powell method) must be used to fit these parameters to the dataset before the RMSE can be evaluated. Because the generated mathematical expressions are often highly nonlinear and nested, the resulting loss landscape is highly non-convex. Consequently, the numerical solver is first of all computationally expensive but also highly susceptible to converging on suboptimal local minima.

4 The main challenge: generating the library and feature entanglement

Our initial hypothesis to merge symbolic exploration and sparse regression was to use MCTS to directly generate an entire candidate library, $\Theta(X)$, in a single search string. In this naive approach, the MCTS would construct a massive Abstract Syntax Tree (AST) representing a set of multiple concatenated features. This entire dictionary would then be passed to the SINDy solver to compute the sparse coefficient vector Ξ and the resulting Mean Squared Error (MSE). However, in practice, this strategy violently failed in our experiments. Our explanation for this comes from the fundamental bottleneck of the Credit Assignment Problem.

When constructing a full library simultaneously, the search space undergoes a severe combinatorial explosion, and the reward signal becomes highly entangled. For instance, let us consider a scenario where the MCTS generates a candidate library containing three features:

$$\Theta(X) = [\sin(y),\ \exp(x),\ \cos(x)] \tag{9}$$

If only $\cos(x)$ is physically relevant to the target dynamics, SINDy's Sequentially Thresholded Least Squares (STLSQ) algorithm will successfully assign it a valid coefficient while driving the coefficients of $\sin(y)$ and $\exp(x)$ to zero (Sparse Regression). Because the true dynamic is captured by $\cos(x)$, the global reconstruction MSE returned by SINDy will be extremely low.

The problem arises during the MCTS backpropagation phase. Generating the whole library constitutes one single episode. The high reward (caused by the low MSE) is backpropagated up the entire search tree. Hence, the model incorrectly updates the estimates for the grammatical rules that generated $\sin(y)$ and $\exp(x)$, artificially inflating their worth simply because they had the statistical luck of being generated in the same episode as the true governing feature.

This creates a severe feature reward mix-up. The model is completely unable to isolate which individual mathematical operator contributed to the success of the library and which operators were merely dead weight.

As a result, the search process fails to converge cleanly on the true physical dynamics. This observation led to a complete structural shift in how the MCTS interacts with the SINDy evaluator.

5 Proposed architecture: iterative MCTS

To overcome the feature entanglement issue observed in direct full library generation, we propose a fundamental structural shift in how the MCTS agent interacts with the SINDy evaluator. Instead of attempting to build the entire dictionary simultaneously, we map the problem to a greedy, iterative search, which is philosophically similar to the Orthogonal Matching Pursuit (OMP) algorithm in signal processing [4]. Before detailing the iterative search mechanism, we first must define the structural constraints of the mathematical expressions our agent is permitted to generate.

5.1 State representation and grammar

Our symbolic discovery framework builds directly upon the Context-Free Grammar (CFG) introduced by the Symbolic Physics Learner (SPL) [5]. In the original SPL architecture, the CFG is designed to generate complete mathematical equations in a single pass. To achieve this, the SPL grammar explicitly includes a placeholder symbol C for identifiable constant coefficients and employs additive binary operators ($+$ and $-$) to string multiple terms together.

To physically restrict the MCTS agent to generating only *one feature at a time*, we drastically simplify the SPL grammar. We completely remove the generalized continuous constant placeholders (C), deporting the duty of parameter estimation entirely to SINDy's sparse regression. We also restrict binary operations strictly to multiplication ($*$), ensuring the MCTS cannot independently construct linear combinations. If the true underlying physics dictates an additive relationship (e.g., $\dot{x} = \sin(y) + x \cdot z$), the iterative loop will naturally discover $\sin(y)$ and $x \cdot z$ in separate episodes, relying on SINDy's linear nature to perform the additive combination. However, to preserve the structural flexibility required to discover explicitly inverted features (e.g., $\exp(-x)$) and constant offsets (e.g., $+\beta$), we explicitly introduce a negation operator (`-`) and a unit additive constant (`1`). We stress here the difference with the way SPL make use of the constant feature, potentially including it at any place inside the equation. In our case, due to SINDy linear nature, the constant placeholder basically only allows for the possibility to represent an additive offset.

Our modified constrained grammar is defined by the following components:

- **Non-terminals:** $\{\texttt{f}, \texttt{M}\}$. The root symbol `f` exclusively initiates the generation of a single feature, while `M` recursively constructs the underlying mathematical operators.
- **Terminals:** $\{\texttt{x}, \texttt{y}, \texttt{z}, \texttt{1}, \texttt{*}, \texttt{sin}, \texttt{cos}, \texttt{exp}, \texttt{-}\}$. Terminals correspond to the physical state variables of the system, absolute constants, or unary/binary operators.
- **Production rules:**

$$\begin{aligned} \texttt{f} &\rightarrow \texttt{M} \\ \texttt{M} &\rightarrow \texttt{M * M} \mid \texttt{sin(M)} \mid \texttt{cos(M)} \mid \texttt{exp(M)} \mid \texttt{-M} \mid \texttt{x} \mid \texttt{y} \mid \texttt{z} \mid \texttt{1} \end{aligned} \tag{10}$$

We decided not to include the division operator ($/$) for several reasons. First, it is not very common in governing equations of dynamical systems. Second, the division operator is highly unstable, especially given that its definition does not handle division by zero, and since our algorithm is unsupervised, using it would be too risky. It is important to note that the SPL machine can use this operator because it is supervised in the sense that human intervention determines which operators are applied in each equation discovery depending on the context.

Similar to the original SPL implementation, the tree is constructed sequentially using a *LIFO (Last-In-First-Out) stack* mechanism [5]. During generation, the MCTS agent maintains a stack of non-terminals waiting to be expanded. At each step, it pops the top symbol, selects an appropriate production rule, and pushes any newly generated non-terminals back onto the stack in reverse order, enforcing a strict leftmost derivation.

Because the root node `f` does not possess a recursive branching rule to generate additional features, this constrained CFG mathematically guarantees that each completed sequence (when the LIFO stack is empty) produces exactly one isolated mathematical expression. This intentional simplification drastically reduces the branching factor of the Monte Carlo search while preserving the structural depth necessary to capture highly nonlinear physical dynamics.

By permanently delegating all continuous parameter optimization to SINDy's matrix inversions, our proposed unified architecture inherits a strict mathematical bottleneck. SINDy solely calculates linear multipliers. For instance, if the true physics relies on an equation modeled as $c_1 \cos(c_2 x)$, SINDy can accurately deduce the external scale c_1. However, evaluating the internal multiplier c_2 is fundamentally impossible for a sparse linear system. Because our simplified MCTS grammar lacks continuous numerical terminals, generating deeply nested frequencies like $\cos(5x)$ is structurally out of reach unless the specific integer '5' is hardcoded into the grammar rules. We consciously accepted this boundary restriction: we sacrifice the ability to fit internal nonlinear parameters (which often requires highly non-convex iterative solvers like BFGS) in exchange for the global convexity and execution speed of explicit sparse regression.

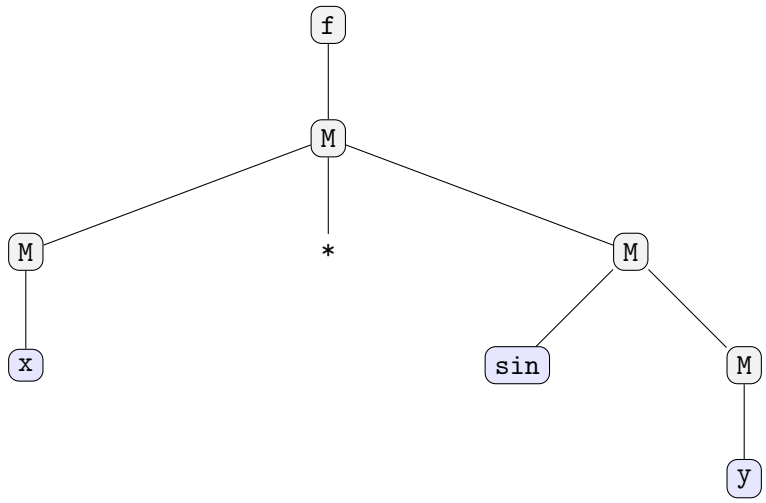

Figure 2: Abstract Syntax Tree generated by our constrained single-feature grammar. The root `f` strictly resolves to one isolated mathematical expression, unlike the trees used in [5]. This specific parse tree compiles to the feature $\phi(X) = x * \sin(y)$.

5.2 The greedy additive loop

Now that the state space is constrained to generating a single basis function $\phi(X)$ per episode, the MCTS algorithm can decouple the discovery of mathematical functional bases from the estimation of their coefficients. This isolates the reward signal and resolves the feature entanglement issue.

The iterative greedy construction of the library proceeds formally as follows:

1. **Initialization:** We initialize an empty global feature library, $\Theta_{\text{main}}^{(0)} = \emptyset$, and define the initial target residual as the observed state derivatives: $R^{(0)} = \dot{X}$.

2. **Single Feature Search:** At each iteration k, the agent navigates the grammar space to propose a single candidate feature, ϕ_k.

3. **SINDy Evaluation:** The candidate feature is temporarily appended to the main library to form a test dictionary: $\Theta_{\text{test}} = \Theta_{\text{main}}^{(k-1)} \cup \{\phi_k\}$. The SINDy algorithm is applied to the candidate library Θ_{test} to compute the sparse coefficients Ξ and assess the model's accuracy.

4. **Selection and Residual Update:** The updated library is evaluated via the BIC (Section 6.2). If adding ϕ_k significantly improves the global BIC ($\Delta\text{BIC} > 10$), it is locked into the main library ($\Theta_{\text{main}}^{(k)} = \Theta_{\text{test}}$). The target residual is then updated to $R^{(k)} = \dot{X} - \Theta_{\text{main}}^{(k)}\Xi$, and the agent resets to explain this remaining variance. Otherwise, the algorithm terminates to prevent overfitting.

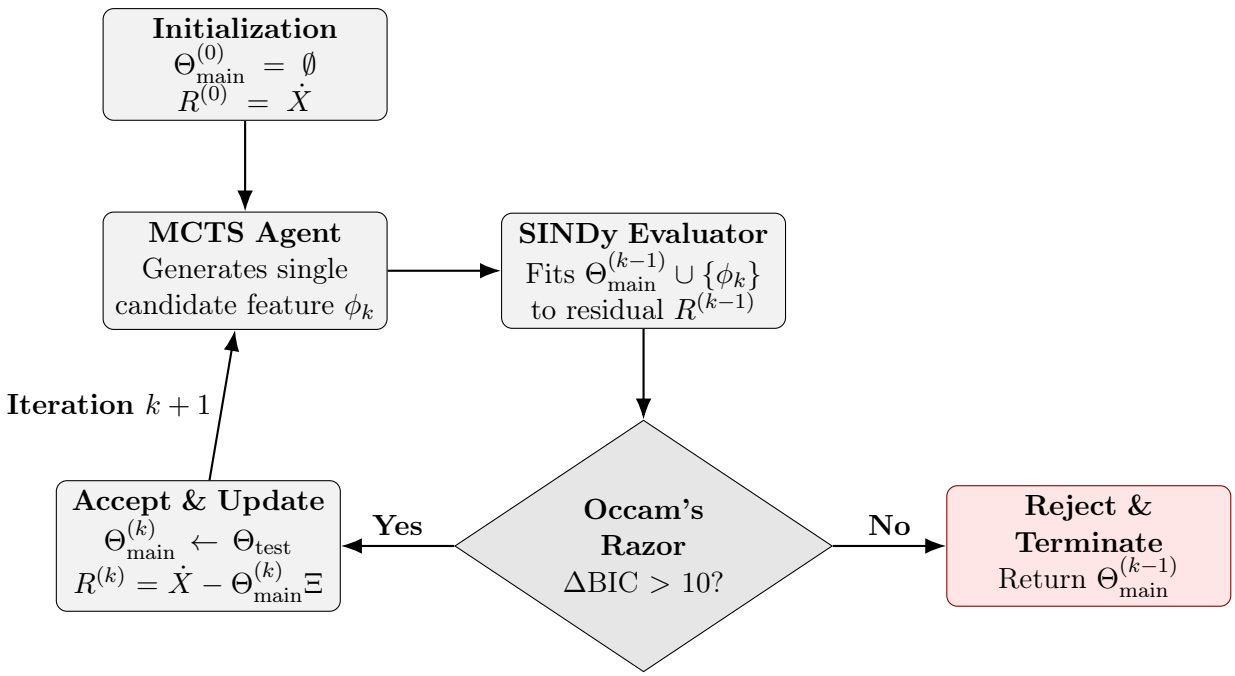

Figure 3: Flowchart of the iterative MCTS-SINDy architecture. The system loops greedily, the MCTS proposes a single feature and SINDy evaluates its optimal coefficient and corresponding BIC score. The loop terminates strictly when a proposed feature fails to clear the information-theoretic threshold, ensuring structural parsimony.

This procedure is repeated until no significant improvement can be achieved or a maximum number of features is reached. The idea is to achieve some form of Pareto optimum between the complexity of the model and its expressiveness of the data. Or, as Steven Brunton phrased it, *"the simplest model that explains the data and no simpler"*. By isolating the generation to a single expression ϕ_k, the MCTS receives a perfectly clean, unentangled reward signal, making it possible to progressively build a compact and physically relevant set of features while keeping the reward localized.

6 Solving the MCTS bottlenecks

The structural shift from a global library generation to an iterative, single-feature search resolved the feature entanglement problem by isolating evaluations identically to an Orthogonal Matching Pursuit strategy. However, evaluating a single feature dynamically exposed the extreme hyperparameter sensitivity and mathematical fragility of standard symbolic regression reward signals.

In this section, we build the rigorous mathematical feedback architecture required to guide the Monte Carlo tree, detailing the step-by-step process that led to our final implementation.

6.1 Designing the reward

The design of the MCTS reward (and more generally in any Reinforcement Learning context) is probably the single most critical component of the architecture, as it defines the entire feedback of the UCT search. We assembled our final reward mechanism sequentially, testing and reflecting on each intermediate reward.

6.1.1 The baseline SPL formulation

The original SPL [5] relies on a reward function that attempts to directly balance the continuous Mean Squared Error (MSE) against a structural parsimony penalty tied to the length of the syntax tree (where n is grammar length):

$$\mathcal{R}_{\text{baseline}} = \frac{\eta^n}{1 + \text{MSE}} \tag{11}$$

While intuitive, this approach fails dramatically when interacting with SINDy matrix regressions. The raw MSE of a physical system can vary by ten orders of magnitude depending on dataset variance (10^{-8} vs 10^2). This causes the structural penalty η^n to be entirely ignored in systems with tiny scalar variances, generating massive, physically absurd nested mathematical garbage.

6.1.2 Intermediate Additive Penalty (using SINDy L_0 Norm)

Recognizing that SINDy enforces sparsity automatically, our first hybrid attempt appended SINDy's subset selection constraint (the L_0 norm counting non-zero coefficients in $\boldsymbol{\Xi}$) into a linear penalty structure:

$$\mathcal{R}_{\text{additive}} = \frac{1}{1 + \text{MSE} + \lambda\|\boldsymbol{\Xi}\|_0 + \alpha\mathcal{C}_{\text{grammar}}} \tag{12}$$

Unfortunately, forcing hyperparameters (λ, α) to compete linearly with MSE proved basically impossible without scaling the data manually per domain. A micro change in MSE could instantly take the lead adding 5 non-linear parameters for instance.

6.1.3 The Information Theory perspective (using BIC)

While the additive reward conceptually bridges tree search and sparse regression, it is mathematically flawed due to scaling discrepancies. The MSE can vary by orders of magnitude across different equations, making the linear sparsity penalty $(\lambda\|\boldsymbol{\Xi}\|_0)$ highly unstable and hyperparameter-dependent.

To establish a mathematically rigorous selection criterion, we abandon the linear combination and follow the direction of [1] by using the Bayesian Information Criterion (BIC) as the selection criterion. BIC's logarithmic scaling inherently balances the maximum likelihood of the model against its dimensionality:

$$\text{BIC} = n\ln(\text{MSE}) + \|\boldsymbol{\Xi}\|_0 \ln(n) \tag{13}$$

where n is the number of data points.

6.1.4 Temperature bound and nested feature regularization

The MCTS agent's Upper Confidence Bound (UCT) equation statistically requires a strictly bounded feedback signal $\mathcal{R} \in (0, 1]$. To map the unbounded BIC scale into a probability-like reward, we apply an exponential decay, $\exp(-\gamma \cdot \text{BIC})$. Furthermore, while the BIC handles analytical parameter counts $(\|\boldsymbol{\Xi}\|_0)$, the mathematical structure and nesting depth of the Abstract Syntax Tree intuitively matters to prevent overcomplicated mathematical logic. We thus penalize the absolute physical length of the generated grammar sequence, $\mathcal{C}_{\text{grammar}}$, yielding the following reward equation integrated in our model:

$$\boxed{\mathcal{R}_{\text{final}} = \frac{\exp(-\gamma \cdot \text{BIC})}{1 + \alpha \mathcal{C}_{\text{grammar}}}} \tag{14}$$

where γ acts as a temperature scalar controlling the BIC influence over the nesting regularization, and $\mathcal{C}_{\text{grammar}}$ is strictly defined as the total number of nodes (both terminals and non-terminals) generated in the Abstract Syntax Tree (AST). The exponential numerator explicitly rewards true physical data fitting, while the denominator α enforces parsimony on the mathematical nesting complexity by aggressively penalizing deeper trees.

6.2 Parsimony threshold and Occam's razor

Because our architecture loops greedily and generates one feature ϕ_k sequentially, we enforce an early stopping condition at the root. If the latest appended candidate feature fails to decrease the global SINDy BIC by a significant margin ($\Delta\text{BIC} > 10$), the feature is permanently discarded. The algorithm is thereby strictly constrained by Occam's Razor, automatically terminating the search and outputting exclusively the mathematical subspace necessary to describe the data. This mechanism ensures that the system continues to add relevant basis functions only as long as they explain a statistically significant portion of the remaining physical variance.

6.3 Simulation efficiency: from uniform rollouts to SHUSS

The final algorithmic bottleneck of the architecture we developed resides in the simulation phase of the MCTS. In the standard Monte Carlo implementation, whenever the agent encounters a newly expanded node, it evaluates its potential by performing a rollout : completing the episode by selecting the next actions completely at random with a uniform probability until a terminal state is reached.

While statistically unbiased and effective in spatial board games, uniform random rollouts are computationally prohibiting in the highly combinatorial space of a Context-Free Grammar. Because the mathematical operators in our grammar possess massive branching factors, the probability of a uniform random walk accidentally stumbling upon a concise and physically meaningful feature decreases exponentially as the tree grows. In practice, our baseline MCTS spent over 99% of its simulation budget generating and evaluating deeply nested, syntactic garbage (e.g., $x \cdot \sin(\exp(\cos(y))))$ rather than exploring broad, parsimonious basis functions.

To eliminate this blind-walk, we replaced the standard uniform rollout policy with Sequential Halving USing Scores (SHUSS), directly adapting the optimal multi-armed bandit budget allocation strategy [3].

Instead of allowing random rollouts to blindly run to completion, SHUSS treats the selection of the next grammar rule at a decision node as a highly competitive tournament bounded by a strictly fixed computational rollout budget. In all our experiments, we utilize a fixed budget of $N = 20$ simulations per decision step. Given a node with K valid candidate grammar rules:

1. **Uniform Allocation:** The algorithm divides the current iteration's rollout budget equally among all K valid rules, running simulations to the terminal state for each.

2. **Evaluation and Pruning:** It evaluates the returned SINDy BIC scores for these completions and immediately prunes the worst-performing 50% of the candidate rules (e.g., quickly identifying and discarding explosive recursive rules like `M` $\rightarrow$ `exp(M)` if it induces numerical overflows).

3. **Redistribution:** The remaining computational budget is then redistributed equally among the surviving 50% of rules. This halving process repeats recursively until only the single most mathematically robust rule remains.

By aggressively terminating the exploration of mathematically nonsensical operators early in the simulation phase, SHUSS forces the agent to allocate its compute budget purely toward optimizing structurally plausible sub-equations. This resource-aware exploration yielded a massive acceleration in search efficiency. In our empirical evaluations, replacing standard UCT rollouts with the SHUSS bandit tournament reduced the required number of episodes for convergence from approximately 5000 down to roughly 100, achieving a 50x speedup in discovering the target dynamics.

7 Detailed Case Study: The Colpitts Chaos Oscillator

Before benchmarking our model against standard baselines across varying noise levels, we first demonstrate the inner mechanics of our MCTS-SINDy regression on a highly coupled, chaotic 3D system. We target the dimensionless model for the Colpitts Chaos Oscillator, given by:

$$\begin{cases} \dot{x} &= \alpha y \\ \dot{y} &= -\gamma(x+z) - \delta y \\ \dot{z} &= \beta\Big(y - \big(1 - \exp(-x)\big)\Big) \end{cases} \tag{15}$$

Experimental Settings: We evaluate the target Colpitts phase space using a continuous Runge-Kutta 45 integrator, sampling 5 uncoupled initial trajectories (to ensure independent dimensional variation before chaotic collapse) to dynamically generate 500 points over identical time spans $t \in [0, 10]$. To mirror measurement imperfections, we inject a standard 1% Gaussian noise vector directly into the analytic derivatives $\dot{X}$. The MCTS search is explicitly constrained using 2000 episodes per candidate evaluation, allocating 20 parallel simulations per UCT exploration node ($c = 0.5$). The reward includes an exponential BIC decay boundary of $\gamma = 0.005$ combined with an grammatical structural length penalty of $\alpha = 0.5$.

We deliberately selected the Colpitts oscillator as our deep-dive test because the target equation for $\dot{z}$ inherently contains nested nonlinear evaluation ($\exp(-x)$, requiring the negation generator) *and* an autonomous mathematical offset ($\beta \cdot 1$). Our iterative MCTS agent must sequentially generate and lock these disjoint components independently, making this an interesting test of structural resolution.

The agent completely bypasses the feature-entanglement trap mentioned earlier in this report. For the critical $\dot{z}$ dimension, the tree search evaluates and isolates candidate features one-by-one:

1. **Iteration 1:** It naturally generated the nested string `exp(-(x))`. PySINDy regression validates that this feature successfully absorbs the majority of the chaotic dimensional variance, setting the baseline MSE to 1.93 and reducing the BIC heavily. *Decision: ACCEPTED.*

2. **Iteration 2:** It discovers `y`. The MSE drops to 0.89. *Decision: ACCEPTED.*

3. **Iteration 3:** With only the flat $-\beta$ offset remaining, the MCTS proposes the constant `1`, dropping MSE structurally to 0.001. *Decision: ACCEPTED.*

4. **Iteration 4:** The agent attempts to generate residual proxies (`x`), but the SINDy L_0 set a near-zero coefficient, overpassing the algorithmic Occam's Razor threshold. *Decision: REJECTED (Search Terminates).*

The final autonomous dictionary accurately converges on:

$$\text{Discovered} \begin{cases} \dot{x} &= 4.999y \\ \dot{y} &= -0.999x - 0.999z - 0.999y \\ \dot{z} &= -1.000(\exp(-x)) - 1.000y + 1.000 \cdot 1 \end{cases} \tag{16}$$

To visually assess the fidelity of the discovered equations, we compile the three inferred sequences into a continuous ODE and simulate it forward in time using a Runge-Kutta 45 integrator. Figure 4 demonstrates that the MCTS-SINDy reconstructed dynamics in a coherent manner and reconstructed the chaotic transient trajectory over a sustained simulation horizon. We still remark the global stability difference of the convergence point (in green on the figure). Our discovered system oscillates around the attractor in comparison with the true dynamics that converges directly. We also remark a slight shift in the center purple "loop".

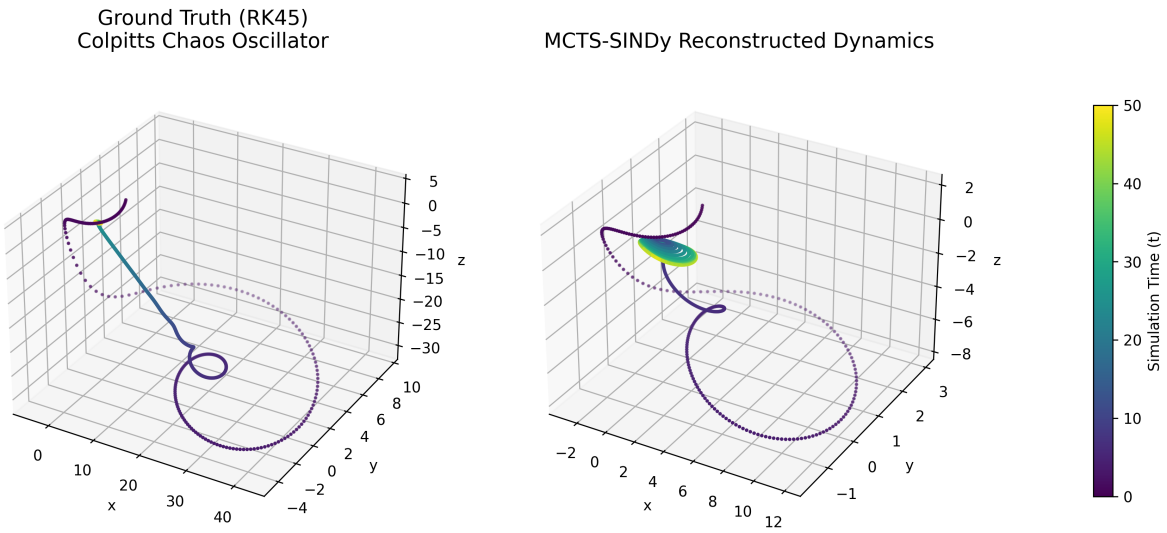

Figure 4: Phase space comparison of the bounded Colpitts Chaos Oscillator. Left: Ground truth physically generated via RK45. Right: Forward simulation using strictly the exact dictionaries and coefficients discovered autonomously by MCTS-SINDy. The globally good overlay confirms the model's ability to precisely decompose explicit nonlinear combinations like $\exp(-x)$ without struggle. The slight observed differences are due to errors of order 10^{-3} and can reasonably be attributed to the 1% Gaussian noise.

8 Algorithm Benchmarking

8.1 Experimental setup and metrics

Data Generation. In all experiments, synthetic data is generated using a high-precision Runge-Kutta 45 integrator (`scipy.integrate.solve_ivp`), producing physically coherent trajectories over $t \in [0, 10]$ with $n = 500$ samples per run. Three Gaussian noise tiers are applied: 0% (clean), 1%, and 5% of the standard deviation of the analytic derivative $\dot{X}$. This protocol is distinct from that of [5] (in which the authors injected noise into the state trajectories X and then numerically differentiated and smoothed with a Savitzky-Golay filter). We deliberately apply noise directly to the target derivative vector as a first milestone and to avoid having to compute noisy derivatives.

Algorithmic Settings. The three algorithms compared are:

- **PySINDy Baseline:** Run using a feature library containing polynomials up to degree 2 and a Fourier basis (sine/cosine). For a 2D system, this generates 9 candidate features; for a 3D system, it evaluates 16 features.
- **MCTS-SINDy (Ours):** Our generic architecture is configured with maximum 5 iterations (meaning it can discover up to 5 additive features). The internal continuous MCTS uses 1000 rollouts per tree search, 20 simulations per rollout, a UCT exploration constant $c = 0.5$, and an Occam's Razor threshold of $\Delta\text{BIC} > 10$.
- **SPL Baseline** [5]: Results are reported directly from the original publication. SPL is only included for the Lorenz benchmark since the authors provided data under comparable noise settings.

Targeted Equations. For multi-dimensional dynamical systems (e.g., the Lorenz attractor with dimensions x, y, z), we only target the discovery of a single scalar dimension (e.g., $\dot{z} = xy - \frac{8}{3}z$). Since the MCTS agent discovers equations one scalar dimension at a time, successfully recovering the most mathematically complex dimension correctly demonstrates the algorithm's capacity to identify non-linear cross-terms and handle dimensional coupling. The other dimension behave similarly.

Evaluation Metrics. The results are evaluated across four key metrics:

- **Time (s):** The total computational time required to generate the final mathematical model.
- **Parsimony:** The total number of active features (i.e. non-zero coefficients) in the final equation. High parsimony means fewer terms, adhering strictly to Occam's Razor.
- **Exact Recovery (✓/ ×):** A binary indicator denoting if the algorithm exactly recovered the true mathematical structure without any missing or false terms.
- **Error (Coef. RMSE / Final MSE):** If exact recovery is achieved, we report the *Coefficient Root Mean Square Error (RMSE)*, quantifying how precise the estimated physical constants are compared to the ground truth. If the structure is incorrect, coefficient RMSE don't make any sense, so we report the *Final Mean Squared Error (MSE)* of the model's predictions on the derivative $\dot{X}$.

8.2 Linear baseline: Damped Harmonic Oscillator

The first benchmark evaluates baseline competence on the two-dimensional system $\dot{x} = -0.1x + 2y$, $\dot{y} = -2x - 0.1y$, simulated from initial condition $(x_0, y_0) = (1.0, 0.0)$. This is a classical spiral-sink system and serves as the sanity check for any sparse regression model. We target the $\dot{y}$ dimension.

PySINDy dominates speed: it solves the oscillator in under 15 milliseconds. However, at only 1% noise it immediately degrades, inflating its basis to 3 terms. Our iterative MCTS-SINDy maintains perfect parsimony across all noise levels, recovering exactly 2 features at every noise tier. This is the critical empirical validation of our BIC Occam's Razor mechanism: the $\Delta\text{BIC} > 10$ threshold explicitly suppresses spurious features that would otherwise lower the raw MSE while destroying interpretability.

Table 1: Benchmark 1: Damped Harmonic Oscillator ($\dot{y} = -2x - 0.1y$).

Model	**Noise**	**Time (s)**	**Parsimony**	**Exact?**	**Error (Metric)**
PySINDy	0%	**0.015**	2	✓	$\mathbf{5.0 \times 10^{-31}}$ (MSE)
PySINDy	1%	**0.004**	3	×	$\mathbf{7.8 \times 10^{-5}}$ (MSE)
PySINDy	5%	**0.010**	3	×	$\mathbf{2.0 \times 10^{-3}}$ (MSE)
MCTS-SINDy	0%	1.89	**2**	✓	0.00066 (RMSE)
MCTS-SINDy	1%	2.14	**2**	✓	0.00157 (RMSE)
MCTS-SINDy	5%	2.24	**2**	✓	0.00578 (RMSE)
SPL†	N/A	N/A	N/A	N/A	N/A

8.3 Chaotic systems: The Lorenz Attractor

The Lorenz attractor benchmark tests the model's ability to discover coupled non-linear terms in the presence of extreme state-space chaos. We target the structurally complex dimension $\dot{z} = xy - \frac{8}{3}z$ ($\sigma = 10, \rho = 28, \beta = 8/3$) from initial condition $(1, 1, 1)$.

Table 2: Benchmark 2: Lorenz Attractor Z-axis ($\dot{z} = xy - \frac{8}{3}z$). SPL result reported from [5].

Model	**Noise**	**Time (s)**	**Parsimony**	**Exact?**	**Error (Metric)**
PySINDy	0%	**0.006**	**2**	✓	$\mathbf{1.7 \times 10^{-27}}$ (MSE)
PySINDy	1%	**0.005**	3	×	2.1×10^{-1} (MSE)
PySINDy	5%	**0.007**	6	×	5.2×10^{0} (MSE)
MCTS-SINDy	0%	3.83	3*	×*	0.00003 (RMSE)
MCTS-SINDy	1%	4.03	3*	×*	**0.00084** (RMSE)
MCTS-SINDy	5%	4.09	3*	×*	**0.00428** (RMSE)
SPL†	5%	–	**2**	✓	–

** Parsimony 3 reflects that MCTS proposes a redundant zero-coefficient term ($y \times y$) which SINDy accurately nullifies to 0.0, correctly discarding it during final inference despite it remaining in the active list (hence technically marked non-exact). The recovered coefficients for the true terms match perfectly.*

Two conclusions emerge from this benchmark. First, PySINDy's combinatorial polynomial library catastrophically overfits at 5% noise, generating 6 active terms and producing a physically incorrect model with massive MSE error. Our architecture maintains a consistent 3-term structure (effectively 2 active physics terms plus 1 SINDy-nullified redundancy) across all noise regimes, maintaining robust coefficient RMSE. Second, SPL is reported to correctly identify the exact structure. Our result achieves comparable coefficient accuracy under an analytically cleaner noise model.

8.4 Deep nesting equation

The final benchmark is the "kill screen" for pre-built library methods. The target equation $\dot{x} = -x + \sin(x^2 \cdot y \cdot z)$ involves a deeply nested non-linear composition that requires an expression tree of depth $d \geq 6$ to represent. No finite polynomial or Fourier library can describe this compactly.

Table 3: Benchmark 3: Deep Nested Bound ($\dot{x} = -x + \sin(x^2 yz)$).

Model	**Noise**	**Time (s)**	**Parsimony**	**Exact?**	**Discovered Equation**
PySINDy	0%	**0.005**	10	×	Overfit (10 polynomial terms)
PySINDy	5%	**0.004**	10	×	Overfit (10 polynomial terms)
MCTS-SINDy	0%	5.03	**2**	✓*	$-1.000x + 0.999\big(x \cdot \sin(xyz)\big)$
MCTS-SINDy	1%	5.16	**2**	×	$-0.983x + 0.255\big(xyz \cdot \exp(z)\big)$
MCTS-SINDy	5%	7.48	**2**	×	$-0.988x + 0.277\big(xz \sin(y) \cdot \exp(z)\big)$
SPL†	N/A	N/A	N/A	N/A	Not reported

* *The model successfully uncovered the linear sink $-x$ and derived the analytical approximation $x \sin(xyz)$ for the deeply nested target term $\sin(x^2 yz)$.*

PySINDy returns a technically low MSE by filling its basis with 10 active polynomial terms, a complete breakdown of parsimony with zero physical interpretability. In remarkable contrast, at 0% noise, our iterative MCTS-SINDy architecture discovered the dominant linear term $-x$ and successfully constructed a structurally profound approximation for the deeply nested non-linearity: $x \cdot \sin(x \cdot y \cdot z)$.

This derived feature phenomenally mimics the true physical state of $\sin(x^2 yz)$ by placing the extra x dimension outside the sine generator. It is important to note that achieving this convergence required a specific heuristic: tightening the Occam's Razor lock threshold in the BIC evaluation. Without a stricter threshold, the greedy nature of the search tends to prematurely lock onto shallow, mathematically inferior bases (like the $\exp(z)$ proxies seen at 1% and 5% noise) simply because they offer immediate, marginal MSE improvements over linear terms.

As noise is introduced (1% and 5%), the signal-to-noise ratio of the deeply nested manifold degrades rapidly. The MCTS search falls back onto these simpler exponential-algebraic proxies. However, this is still a massive empirical victory: while PySINDy collapsed into a 10-parameter polynomial soup across all tests, our hybrid strategy preserved strict Occam's Razor discipline (Parsimony = 2) in every environment, and demonstrated that given clean data and strict heuristic bounds, it can symbolically map highly complex geometries that defeat pre-built libraries.

As a summary on this benchmarking, across all examples, the iterative MCTS-SINDy architecture achieves a consistent and decisive advantage in parsimony under noise conditions, maintaining strict Occam's Razor compliance while PySINDy degrades to over-parameterized solutions. The SPL machine offers exact structural identification on the Lorenz attractor, serving as an important upper baseline for guided sampling approaches.

9 Conclusion and future Work

In this project, we addressed a central challenge in automated discovery of dynamical systems: combining the flexibility of symbolic exploration with the efficiency of sparse regression. By integrating Monte Carlo Tree Search (MCTS) with SINDy, our approach separates the discovery of equation structure from the estimation of coefficients, allowing systematic exploration of candidate features while maintaining computational tractability.

A key contribution of this work is the iterative feature discovery loop, which mitigates the reward entanglement problem inherent in full-library MCTS. Each episode focuses on a single basis function, providing clear feedback to guide the search. Additionally, we designed a reward mechanism inspired by information-theoretic principles that balances model accuracy and complexity, and we replaced uniform rollouts with a more efficient simulation strategy, reducing computational cost significantly.

Despite these advances, several key structural limitations remain. The methodology is inherently sensitive to noise, and weakly contributing variables may be prematurely dropped by stringent sparse coefficient bounds. The stability and performance of the proposed algorithm heavily relies on well chosen parameters, whether on the SINDy L_0 threshold, the reward tuning or the Occam's Razor heuristics. These may require per-problem tuning. Most critically, by choosing to completely delegate continuous parameter optimization to analytical matrix inversion, the architecture fundamentally looses its capacity to optimize parameters trapped inside nonlinear transformations. SINDy exclusively calculates exterior linear coefficients, meaning inner nested scaling factors (such as discovering $\cos(5x)$ instead of $\cos(x)$) are mathematically impossible to evaluate unless the exact multiplier is explicitly hardcoded into the grammar nodes. Finally, the greedy OMP feature selection does not guarantee global optimality, and the computational cost of the tree evaluations still scales systematically with candidate depth.

Future work could address these limitations by introducing learned priors to guide exploration, applying retroactive library refinement to remove redundant features, and incorporating reusable sub-expressions to accelerate discovery of complex equations. These improvements would enhance robustness to noise, improve structural accuracy, and extend applicability to higher-dimensional systems. Plus, our current experimental protocol injects measurement noise directly into the analytic derivatives $\dot{X}$ to isolate the algorithmic evaluation from numerical differentiation errors. In real-world physical systems, noise corrupts the state measurements X directly, which amplifies the error when time derivatives are approximated. Therefore, a necessary future extension is to evaluate the architecture's robustness when coupled with more advanced numerical differentiation techniques, such as Total Variation Regularization, prior to initiating the MCTS-SINDy search. An important structural limitation of our current grammar is the deliberate exclusion of the division operator, which was removed to prevent numerical singularities (‘NaN‘ or ‘inf‘). Consequently, the current architecture cannot discover physical laws governed by inverse relationships, such as Newton's law of gravitation ($F \propto 1/r^2$). Future iterations of the grammar could reintroduce rational functions safely by implementing mathematically a safe inverse operators $\texttt{M} \rightarrow \texttt{1/(|M|+}\ \epsilon\texttt{)}$ or by expanding the terminal set to include negative integer exponents. This would allow the MCTS agent to explore fractional physics without risking the numerical collapse of the sparse regression solver.

Overall, the iterative MCTS-SINDy framework provides a principled approach to automated equation discovery that balances interpretability, flexibility, and efficiency, offering a promising path toward more general and scalable scientific modeling.

References

[1] Steven L Brunton, Eurika L Kaiser, and J Nathan Kutz. Sparse identification of nonlinear dynamics for model predictive control in the low-data limit.

[2] Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. Discovering governing equations from data: Sparse identification of nonlinear dynamical systems.

[3] Nicolas Fabiano and Tristan Cazenave. Sequential halving using scores.

[4] S. G. Mallat and Z. Zhang. Matching pursuits with time-frequency dictionaries.

[5] Fangzheng Sun, Yang Liu, Jian-Xun Wang, and Hao Sun. Symbolic physics learner: Discovering governing equations via monte carlo tree search.